8/8/26

EXPERIMENT-6

ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM:

To write a program to implement error detection & correction using the Hamming code.

ALGORITHM:

1) Start the program.
2) Read the number of 4-bit data blocks(n) forming the input data stream.
3) For each 4-bit data block D = d1, d2 d3 d4, performing Hamming code.
   a) Arrange 7 bits with parity bits at positions 1,2,4 & bits at positions 3,5,6 & 7.
   b) Compute the parity bits using even parity:
      P1 = d1 XOR d2 XOR d4
      P2 = d1 XOR d2 XOR d4
   c) Form the 7-bit hamming coord: P1 P2 d1 P4 d2 d3 d4
4) At the Receiver, recompute the check bits from the received code word.
5) Compute the error position = c4 c2 c1.
6) If error position = 0, no error has occured.
7) Extend the original 4 data bits from the corrected 7-bit codeword.
8) Stop the program.

PROGRAM:

```
def encode(d):
    """d: list of 4 bits [d1,d2,d3,d4] → returns 7 bit hamming code] list"""
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos} → correcting....")
        h[error_pos] ^= 1
    else:
        print("No error detected")
    return error_pos

def main():
    print("Hamming code - Error detection & Correction")
    h = int(input())
```

```
for b in range(1, n+1):
    print(f"Block {b}:")

d = list(map(int, input("Enter 4 data bit ").
    split()))

h = encode(d)
print(f"Original data bits : {to-str(d)}")
print(f"Enceding hamming code : {to-str(h)}")
pos = int(input())
if 1 <= pos <= 7:
    h[pos-1] ^= 1
    print(f"Transmitted code(with error at pos):
        {to-str(h)}")

else:
    print(f"Transmitted code : {to-str(h)}")

print(f"Received code : {to-str(h)}")
print("checking for errors.....")
detect_and_correct(h)
print(f"corrected code : {to-str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered data bits : {to-str(rd)}")

if __name__ == "__main__":
    main()
```

RESULT:

The program to implement error detection & correction using the Hamming code concept was successfully written in Python, executed & verified using a sample data stream.

OUTPUT:

Enter number of 4-bit data blocks in the stream: 2

---Block 1---

Enter 4 data bits: 1011

Original data bits: 1011

Encoded Hamming code (7 bits): 0110011

Enter bit position to introduce error: 3

Transmitted code: 0100011

Received code: 0100011

Recovered code: 1011

Corrected code: 0110011

---Block 2---

Enter 4 data bits: 0110

Original data bits: 0110

Encoded Hamming code (7-bits): 1100110

Enter bit position to introduce error (1-7, 0 = no error): 6

Transmitted code: 1100100

Received code: 1100100

Error detected at bit position: 6 → correcting...

Corrected code: 1100110

Recovered data/code: 0110